

For this example, let's suppose that an issue occurred in GitLab Rails. To simulate this:

1. The first location that requests reach for this component is Nginx. Let's take a look at the logs by running:

```
bash
kubectl logs <webservice-pod-name>
```

1. In this output, search for your HTTP request. It will look something like this:

```
bash
173.34.175.144 - - [25/Oct/2024:14:48:00 +0000] "GET / HTTP/1.1" 502 2026
"http://34.56.107.198/" "Mozilla/5.0 (Macintosh; Intel Mac OS X 10157) AppleWebKit/537.36
(KHTML, like Gecko) Chrome/129.0.0.0 Safari/537.36" -
```

> We can tell this is the request based on the URL being accessed and the Nginx status code. This status code is a 502 and the request is to the root of the web application.

1. This output tells us our request reaches Nginx. From here, we can trace to the next location in the architecture, Workhorse. To view the logs for Workhorse, run:

```
bash
kubectl logs <webservice-pod-name>
```

1. In this output, look for the request that matches the URL of your Nginx request. It will look like this:

```
json
{"backendid":"rails","contenttype":"text/html; charset=utf-
8","correlationid":"01JB22H7ENN72DH5XNMTB2170Z","durationms":0,"host":"34.56.107.198","level":"
info","method":"GET","msg":"access","proto":"HTTP/1.1","referrer":"http://34.56.107.198/","remo
teaddr":"173.34.175.144:0","remoteip":"173.34.175.144","route":"","routeid":"default","status":
502,"system":"http","time":"2024-10-
25T14:50:49Z","ttfbms":0,"uri":"/favicon.ico","useragent":"Mozilla/5.0 (Macintosh; Intel Mac OS
X 10157) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/129.0.0.0
Safari/537.36","writtenbytes":2026}
```

> In this request, we can collect some additional important details. We can see our request was sent to rails. If the request reached rails, we can use the correlationid to locate it in the logs. Let's take note of the correlation ID and search for it in the Rails log.

1. To view the log for your Rails application, run the command:

```
bash
kubectl logs <webservice-pod-name>
```

1. These logs have a lot of data, since we are looking for a correlation ID, we can try grepping for it in the logs:

```
bash
kubectl logs <webservice-pod-name> | grep your-correlation-id
```

1. You will get no results. This tells you the GitLab Rails did not receive the request from Workhorse. From here, we can diagnose the issue with rails, starting by looking at the status:

```
bash
kubectl get pods
```

Task D. Collecting SOS

In some cases, troubleshooting will require the assistance of GitLab Support. To help GitLab Support troubleshoot any errors, it is helpful to provide a full account of your instance logs. For this purpose, you can use the GitLab SOS tool.

1. To run GitLab SOS, use the following commands:

```
bash
curl https://gitlab.com/gitlab-com/support/toolbox/kubesos/raw/main/kubeSOS.sh | bash -s --
```

1. After the run completes, you will see the location of your support bundle, in the form of `/root/gitlabsos/gitlabsos.your-instance-namegitlab-nginx-psql-puma-redis-sidekiq.tar.gz`.

> When you file support requests, you can attach this bundle to provide all of your instance logs.

Lab Guide Complete

You have completed this lab exercise. You can view the other lab guides for this course.

Suggestions?

If you'd like to suggest changes to the GitLab System Admin Basics Hands-on Guide, please submit them via merge request.

SECTION: customer-success/professional-services-engineering/engagement-mgmt/_index.md

Role of the Engagement Manager

The Professional Services (PS) Engagement Manager is responsible for helping Solution Architects (SAs) and the sales team develop and gain approval for custom Statement of Works

(SoWs). The process typically includes the following activities.

- An initial meeting with the SA or Customer to understand the objective and/or specific requirements
- Develop an initial Estimate based upon the Services Calculator
- Update the Estimate based upon initial feedback from the Customer and/or Sales team
- Once agreed, generate a supporting Statement of Work (SoW)
- Create a cost estimate (COGS)
- Providing clarification to the PS Director in order to obtain approval for the SoW
- Present the SoW to the account team to gain signature
- Track progress of the SoW through to signature
- Provide soft hand-off of the SOW to the Delivery team upon signature to ensure a smooth transition.

Detailed Job Responsibility Prioritization

Meet the Team

Check out our team of Engagement Managers on our team page

How to contact or collaborate with us

- AEs can create a professional services only opportunity in SFDC. This triggers the automatic creation of a scoping issue used for collaboration during PS discovery, scoping and proposal presentation.
- Slack for informal and general questions, coordination
- Salesforce Chatter for approvals, discussion of opportunity status, or collaboration with sales leadership and legal.

Useful Resources

- Services to Accelerate Customer Adoption
- Selling Professional Services
- Professional Services Offering Framework
- Services Calculator
- Certified GitLab Engagement Manager Learning Journey
- Main Customer Slide Template
- Services data sheets

Engagement Manager - Processes

- Tracking Professional Services Opportunities
- Engagement Scoping Information
- SOW Creation
- Quote Creation process for opportunities with both Custom Services and Fixed Price Training on a single SOW
- Transacting ProServ business through AWS Marketplace

SECTION: customer-success/professional-services-engineering/engagement-mgmt/aws-marketplace.md

Processing a PS Opportunity through AWS Marketplace

See internal handbook page for details

SECTION: customer-success/professional-services-engineering/engagement-mgmt/gitlab-certified-engagement-manager.md

Overview

This course provide Professional Services Engagement Managers with the information they need to be able to position, scope, and sell GitLab professional Services. In the course you will learn about the Professional Services value proposition and how it relates to the overall GitLab value proposition. You will gain insights into the processes that we use, the services that we can offer, and the collaboration we expect to have with our account teams and customers.

Availability

This certification is currently available to any GitLab team member. We are exploring ideas of making this available to partner services sellers at a later date. As a GitLab team member, you can find the GitLab Engagement Manager Accreditation Courses on our standard learning platform.

Exam Objectives

Individuals who earn GitLab Certified Engagement Manager Professional certification are able to perform the following fundamental tasks.

- Work with Account Teams to gather information about a customer opportunity
- Position the value of Professional Services
- Perform discovery calls with customer
- Understand GitLab service offerings to be able to build a proposal and cost estimate
- Create Statements of Work (SOW) based on customer-approved estimates
- Gain appropriate approvals to close business

SECTION: customer-success/professional-services-engineering/engagement-mgmt/sow-processing.md

SOW Creation

:warning: This page is under construction

We use the SOW generator that is part of the Services Calculator project to create SOWs. See that project's README for additional information on how to generate an SOW.

SECTION: customer-success/professional-services-engineering/engagement-mgmt/task-prioritization.md

Prioritization of Job Responsibilities

The list of job responsibilities is in ranked order with the highest priority task at the top.

SOW Processing for current opportunities including

- Creating a draft SOW from an estimate and providing to the customer for review
- Working through SOW redlines with a customer in conjunction with legal and the account team
- Submitting final SOW for approvals and legal stamp
- Sending SOW to Dale Brown (Principal Accounting Officer) and customer for signature. Alternatively sending SOW to Brian Robins (Chief Financial Officer) if Dale is unavailable.
- Creating a COGS and uploading to salesforce

Note: SOW processing activities are a collaboration between the EM and the account team; who the "DRI" is will vary based on the circumstances. For smaller, tactical opportunities, the EM might own the entire process with little or no SAE involvement. For larger, strategic opportunities, especially when there are licenses and services being sold together, the SAE might handle all of the communication with the customer themselves. It's important to have a conversation with the SAE for each opportunity to gain agreement on how that opportunity will be worked. It's also important to keep a collaborative mindset when working with the SAEs.

Suggested SLAs:

- 48 hours from confirmation of customer intent to move forward with an SOW to SOW draft in their hands.
- 72 hours from confirmation of final SOW content to stamped SOW in customer's hands for signature (Revenue and PS approval -> Legal stamp -> Dale signature-> Customer)

Scoping of Early Stage PS Opportunities

- Working with SA to get the appropriate information ahead of a scoping call, including reminding them to provide the information in the scoping issue
- Scheduling internal sync meetings as needed prior to scoping calls
- Leading customer scoping meetings
- Work with Technical Architect as needed to drive technical discussions that lead to the appropriate scope, and to review estimates over \$50k
- Providing estimate to account team and customer for feedback

Suggested SLAs:

- 24 hours from initial scoping issue creation to EM interaction/reminder to SA or CSM to provide information about customer need
- 72 hours from customer scoping call to estimate in customer's hands for review

Territory Opportunity Management

- Updating sfdc EM status regularly
- Following up with account teams to have them update opportunity stage, close date, and quote when they are out of alignment with current state
- Providing regular input to current quarter forecast updates
- Regular communication with account teams to understand current state and next steps for current quarter opportunities

Scoping Issue Management

- Ensuring that scoping issues have appropriate Segment, and workflow labels
- Marking closed opportunities with closeqtr and proposal::closedwon or proposal::closedlost labels
- Updating scoping issues with all relevant information from customer calls and internal collaboration

Account Handoff and Delivery Collaboration

- Asking customer readiness questions and documenting them
- Providing information about PS scope, skillset needed for delivery, customer timing expectations for delivery, and customer readiness to PC and PM
- Participating in PM-led internal hand off meetings
- Participating in delivery escalation meetings, as needed
- Collaborating with PM on change orders

Sales/PS Collaboration

- Participating in regular ASM led team calls
- Participating in other Sales leadership calls as warranted
- Helping to enable sales on PS offerings and process
- Working with ASMs to mature sales team PS selling motions, i.e. ensuring that sales teams can articulate the value of PS, understand our offerings, are employing strong sfdc opportunity hygiene practices, and are following our documented PS and deal desk processes

EM Standardization and Enablement Contributions

- Quoting/SOW gen tool evals
- Updating scoping questions, slides, PS Engagement Estimate Templates, web page materials for our services offerings as we refine them
- Reviewing MR's for updates and providing feedback
- Creating and refining SOW activities in Quilt (or PSQuote if we move to that)
- Collaborating with new EM's as they onboard to help them learn the role
- Providing feedback about customer conversations regarding PS needs to Practice Managers so that they can consider new offerings

Channel Team Relationship Management

Skills Development

- Learning about GitLab offerings and how to position them
- Learning about new GitLab features

- Professional Development

Patch-specific responsibilities

- Commercial - improving the automated sales motion
- Pub Sec - working with the Channel and PS teams to understand partners and contracting vehicles available to transact business
- APAC, EMEA - Working with PS leadership, sales leadership, and Channels to establish fair market rates to sell services at competitive prices

General Responsibilities Not Captured Above

- Act in a collaborative manner with account teams, Professional Services team members, and others within GitLab
- Maintain a current understanding of the forecast and risk for each opportunity in the current quarter
- Escalate any blockers and communicate any needs for help via the ps-engagement-mgrs channel

SECTION: customer-success/professional-services-engineering/engagement-mgmt/tracking-opps.md

Tracking Opportunities

Salesforce.com Tracking

Tracking the overall pipeline for Professional Services and individual opportunities is important to the Professional Services team to retain visibility and ensure capacity for future projects.

This importance increases as we approach the end of the financial quarter (FQ), as we typically see a spike in Closed Won deals at this time.

To help this visibility, the Engagement Managers maintain a subset of fields against an opportunity under the Professional Services section. These fields are reviewed and updated on a regular basis to support reporting to Professional Services leadership on the current forecast:

- Engagement Manager - Denotes the Lead Engagement Manager supporting the opportunity
- EM Confidence Rationale - Context and rationale to support whether the EM feels the opportunity will close within the quarter that it is currently forecasted.
- Scoping Issue Link - A direct link to the supporting scoping issue.
- Professional Services Value - EMs should update this field to represent the current expected value of the engagement. This is primarily for forecasting purposes. It is expected that this field is overwritten with the quoted value at the point of quoting. This value is inclusive of all education and consulting services that are paid for by the customer. If you are planning to request investment services, include that value in here until the point of approval on SFDC Chatter. At that point, remove the investment services from Professional services Value and add it to Investment Services Value.
- Preserve Amount - This field is automatically updated with the value of PS after the Account Executive (AE) creates the quote.
- Edu Services Value - This field cannot be edited directly. It will automatically be populated

with the value of the Education services quoted once the quote is created. If multiple quotes are made, the value will overwrite previous values added (including ones that were added by using the below Edu Value Override workflow).

- Edu Value Override - This is an editable field that should be filled in by the EM to accurately forecast early in the opportunity cycle. This will overwrite the value in the Edu Services Value field.
- Investment Services Value - This should be filled in by the EM once Investment services are approved by Sales VP and VP of global services. This will be used for both forecasting and accounting of closed/won investment services.

Scoping Issue Tracking

Scoping Issue EM Boards

Scoping issues are auto-generated by the services calculator in the ps-plan project with the label Services Calculator added to them. Each EM has a board to track scoping issues assigned to them.

Triage of new scoping issues

New scoping issues can be accessed from the "Issues by EM" board and will be tagged with the Open label. Any Engagement Manager can triage new issues, label them appropriately, and assign them to the correct EM. The EM assigned to the scoping issue should be the person who works the region specified in the account record, under the Account Demographics section, Sales Territory field, unless otherwise determined by the team in order to balance scoping issue load across team members.

- Add the appropriate Segment:: label based on which region owns the opportunity (see "Other labels" section below).
- Assign the new scoping issue to the relevant EM who owns this region

The board uses scoped labels with the proposal key to track the standard scoping issue workflow:

Column	Definition
Open	Starting state for each scoping issue - EM has not reviewed the scoping issue yet
proposal::AwaitingDiscovery	EM is waiting for additional information, either from account team, or via a scoping call, in order to be able to create an estimate
proposal::StrawmanWIP	EM has a strawman estimate in process
proposal::EstimateFeedback	EM has completed the strawman estimate and is awaiting feedback from the account team and/or the customer
proposal::Onhold	PS engagement might proceed but there has been a delay in the timing
proposal::SOWWIP	EM is in the process of creating the SOW, or SOW draft is being reviewed by the customer and we are awaiting feedback
proposal::Ready For Approval	SOW has been drafted and reviewed by account team and customer and we have been given the greenlight to proceed for signatures; GitLab revenue has approved
proposal::Approved	SOW has been approved by Dir. of PS and can be submitted for legal stamp and then signatures
Closed	opportunity has closed or we have received word from customer that they will not

pursue PS |

Delivery Team Review and Approval

During proposal::StrawmanWIP stage, its good practice (for anything that is not-straightforward) to bring in a Technical Architect to review the initial estimate. You can do this by adding a label and signaling in slack. Specific process is defined on the TA Handbook page here.

During the proposal::SOWWIP stage, it is required to gain Regional Delivery Manager (RDM) approval as the RDM is responsible for succesful delivery of the proejct being scoped. You can do this by adding the RDM::needs-review label on the issue and signaling in the ps-engagement-mgrs channel to @ps-deliverymanagers.

Win Rate

Win rate is calculated by:

> Win Rate = ClosedWon / (ClosedWon + ClosedLost)

Where:

- ClosedWon is determined by the associated SFDC opportunity going into closed/won status. This is done with the proposal:ClosedWon label
- ClosedLost is determined by the associated SFDC opportunity going into closed/lost status OR the scoping issue not having any activity for a period of 2 months. This is done with the proposal:ClosedLost label. This helps us be objective with what should be considered Closed/Lost. This is a general principle and can be overridden by an EM if the opportunity is still active, but has not had contribution within the last two months.

As we have a quarterly reporting cycle, we need to know the quarter in which these opportunity scoping issues closed. We accomplish that by adding the closeqtr::QXYY (example:closeqtr::Q322) label to any issue that goes to closed won or lost.

Other Labels Used for Tracking Scoping Issues

- EMStatus - manually added to scoping issues as a visual to see what the EM needs to follow up on
 - Waiting: EM is waiting for information from the account team or customer before we can take action to move the issue to the next stage
 - ToDo: EM has the next action for the current scoping issue
- Segment - manually added to scoping issues to track what sales region the issue is for
 - EntAPAC
 - EntEAST
 - EntWEST
 - EntEMEA
 - MMAPAC
 - MMEast
 - MMEMEA
 - MMWest
 - SMB

Note Public Sector US is a label automatically added to scoping issues that are created via the Services Calculator when the PubSec US checkbox is checked.

SECTION: customer-success/professional-services-engineering/engagement-mgmt/scoping-information/_index.md

Scoping Information

We use the Engagement Estimate TEMPLATES workbook as a starting point for determining the activities, duration, and price of a Professional Services opportunity.

Professional services opportunities with estimates totaling more than \$50,000 USD require that the scoping issue be reviewed by a Technical Architect before an SOW is created. The Engagement Manager should add the TA::Needs-Review label to any scoping issue with an estimate greater than \$50,000. Once the TA has reviewed the estimate, they will change the label to TA:Approved to indicate that they have reviewed and approved the scoping issue.

When an Engagement Manager needs help with scoping due to complex technical requirements or other factors that need technical input, they can add the Scoping::NeedsTAAssistance label to the scoping issue. This label can be used to request that a TA join a scoping call or provide input on a technically complex solution proposal. A Technical Architect's assistance should be requested for any scoping issue or conversation related to GitLab architecture, assessing performance issues for a current implementation, requests for monitoring, or other technical implementation topics.

Applying this label will add the scoping issue to the TA Board. The Engagement should also signal in slack, by mentioning @ps-ta that help is being requested, and provide additional details for the TAs to follow up on.

Once a TA has provided the requested assistance, they will change the label to Scoping::TAAssisted. Using these labels consistently will provide better visibility into the opportunities that currently require TA assistance, as well as historical data on the opportunities that have required TA involvement.

Scoping Specific Types of Services

See the detailed notes pages for each service type below.

- GitLab implementation scoping
- GitLab Readiness Assessment (Health Check)
- GitLab migration scoping - GitLab, Bitbucket Server, or GitHub (Enterprise or .com) -> GitLab
- CI pipeline migration scoping
- Transformational Services scoping

Escalations in Scoping

There may be times when the EM and/or TA disagree with a solution approach that a customer has proposed for Professional Services. In these cases, the scope should be escalated with the

account team (SAE/AE, SA, CSM) by following these steps:

1. Schedule an internal call with the account team (SAE and CSM or SA depending on who you've been working with on scoping, possibly both). Both the TA and EM should attend the internal call.

1. Lay out your reasoning for thinking the current solution path is not the correct path

1. Define alternate solutions that would be a better fit

1. Agree with account team on a communication plan with the customer, next steps, and the DRI for communication

SECTION: customer-success/professional-services-engineering/engagement-mgmt/scoping-information/ci-pipeline.md

General discovery questions to determine customer maturity level

Its helpful to understand the level of maturity that the customer is on as it relates to CI/CD, Containers and repository hygiene. The following questions should be considered if a customer is interested in GitLab Ultimate or the DevOps Platform Concent. Also, consider these questions if conversations on SCM Migration are expanding to migration of additional data elements from the later stages of the DevOps lifecycle.

| Question | Answer | Sample Answer | Rationale for asking|

| ---- | ---- | ---- | ---- |

| How do you use containers in your software development lifecycle today? | | We deploy our software applications into containers OR We use general purpose containers to run CI/CD jobs OR We use purpose-built containers with a tightly managed bill of materials to run CI/CD jobs. | This question is meant to purposely differentiate deploying software as containers vs. using containers to run CI/CD jobs. |

| How do you control/manage the software that is installed and runs on your CI/CD compute environment? | | We allow each application team to install software on their VMs/Containers so they can build/test/scan/deploy their apps. OR We centrally manage the base VMs/Containers that are allowed to perform build/test/scan/deploy jobs. App teams can add software to those VMs/Containers. | We are hoping to understand how the customer manages their SDLC Supply chain and the risks associated with losing control of the software running in their CI/CD ecosystem. |

| How long does it take you to pull in containers into your environment? | | We pull containers from the internet monthly and scan and publish internally. If we need a new container, you can expect this to take about a week for the request to approved, and processed. OR We dont have a scanning approach today; App teams can use whatever container they want. | We want to understand the security practices in place today to ensure we scope services properly. |

| How do you structure your repositories, structured (single tech stack focused on a single solution) vs. unstructured (many tech stacks that host many software solutions) | | Most of the git repositories are focused on a single software solution. We may have a few unstructured repositories, but these are outliers. | We ask this because using strutured repositories will help make use of GitLab CI/CD (and specifically Pipeline COE) out of the box. |

| How do you currently import packages such as Debian or Red Hat packages into your environment today? | | We utilize Red Hat Satellite to handle this. We currently allow developers to import packages themselves. We have an internal Apt/Yum Repository that we import packages into | We

want to understand the level of control/standardization the customer currently has and what level of control/standardization they want to achieve in their future state. |

General approaches for CI/CD "Migration"

1. Flash Cut - Migrate all data from legacy CI platform to GitLab including job definitions, job outputs and artifacts. Note: This is typically harder to do as you will need to recreate history on the new system. Most customers are not interested in the additional cost (development time to script/test, longer migration). Wherever possible, we should recommend Net New.
2. Net New - Update all pipelines job definitions to run on the customer GitLab instance using GitLab CI. All job history remains on the legacy system. All historical packages and containers are left on the legacy system and rebuilt on the new GitLab system.

Migrating from Artifactory to GitLab Registry

Specific Approaches

1. Flash Cut - Migrate all data from Artifactory to GitLab registry including historically built packages and containers. (Not recommended)
2. Net New - Update all pipelines to push to and pull from GitLab registry so that all newly built software packages are stored in GitLab in the build/package stage. Then in the deploy stage they are pulled from GitLab registry. Slowly migrate the dependencies that don't often change to GitLab registry. Gradually phase out Artifactory. (Recommended)

Scoping Questions - Artifactory Migration Opportunity

Question	Answer	Sample Answer	Rationale for asking
What artifacts are stored in artifactory?		built software binaries and docker containers.	
No dependent libraries - we go out to the internet for these	This will affect the decision on general approaches listed above		
How many artifacts are stored in artifactory?		1,234	Shows roughly how large the scope of a migration project will be.
How many artifactory servers do you have?		2	This is a gauge of complexity. The more artifactory services, the more complex.
How many software packages/images are pushed to artifactory per Month?		2345	This shows the customer is measuring this data to be used during the transition process.
How many software packages/images are pulled from artifactory per month?		2345	This shows the customer is measuring this data to be used during the transition process.

Migrating from Jenkins to GitLab CI

Specific Approaches - Jenkins Migration

1. Flash Cut - Migrate all data from Jenkins to GitLab CI and update pipeline files at once.
2. Net New - Update all pipeline job definitions to run on the customer GitLab instance using GitLab CI. All job history remains on the legacy system. All historical packages and containers are left on the legacy system and rebuilt on the new GitLab system.

Scoping Questions - Jenkins Migration Opportunity

Question	Answer	Sample Answer	Rationale for asking
How many Jenkins Instances do you have? Please list them and answer the below questions for each instance.	2 instances - US east and US West		We need to know this to understand the scope of migration
Where are these instances located?	AWS US East 1c, In our on-prem Sacramento data center		Thinking through connectivity between Jenkins servers and GitLab
Total number of jobs?	6,400		This is a gauge of scale and an indicator of project duration
Total number of users?	900		This is a gauge of scale and an indicator of project duration
Total number of repositories?	6,400		This helps us understand cardinality (e.g. how many jobs are associated with each source code project). Its also an indicator of scale of an engagement.
How does Jenkins currently communicate with your SCM?	A single service account is used for polling the SCM for changes. If there is a shift in responsibility (from platform team to application team), this tells us whether we should discuss education services on how to configure the GitLab project.		
What Jenkins plugins are you using most often?	We use the maven plugin extensively. Also some teams use Ant/Gradle plugins. Some usage of .NET SDK too.		Knowing more about the plugins that support your workflow will help us understand how much time to spend on building workflow proposals to use an approach based on GitLab best practices.
Do you use the branch/source plugin to automate the creation of jenkins folders or jobs?	We don't use the branch/source plugin for automatically creating jenkins jobs on repo creation		This automation is usually something customers want to retain, so we will need special considerations for this kind of approach.
What type of job executors pick up your Jenkins jobs?	We have a persistently running fleet of virtual machines that are available to run jobs.		This helps us understand the level of effort to port your workflows to use GitLab best practices (leveraging containerized builds).

SECTION: customer-success/professional-services-engineering/engagement-mgmt/scoping-information/implementation.md

:warning: This page is under construction

Implementation Scoping Details

- We have scoped several different reference architectures in the past. Examples can be found in the Deployment Equations tab of the Engagement Estimates TEMPLATES document.
- Implementations will use the standard documented reference architectures.
- All implementation services on prem with Geo included assume that we use a single large omnibus implementation for the secondary. There are some challenges with full HA secondaries, so if we choose to scope an HA Geo, the general rule of thumb is to add an additional 20-25

days to the scope to account for the complexities.

- Implementations on AWS tend to be less complex than on-prem implementations because we take advantage of the various services AWS provides for HA.

- K8s implementations are generally the same effort as VM implementations, for the same sized reference architecture.

- GitLab System Administration Basic and GitLab Advanced System Administration training classes are typically included in the project scope for implementations. These courses provide foundational concepts for configuring, administering, and troubleshooting GitLab for both single node Omnibus and HA implementations. At this time, we do not recommend these classes for any Kubernetes implementation as the course content does not reflect the kube-specific commands that users need to run for Kubernetes implementations. Customers will get "trained" on backup/restore and upgrade activities during the standard implementation activities. Additional consulting days/post-deployment Q&A sessions can be added to scope to help the customer ramp on any administration topics.

- Our PSE team uses infrastructure as code automation for provisioning infrastructure and configuring the provisioned servers. The tooling is managed in the Proliferate project. The project readme has information on what infra as code we currently support. Note, this is different than the open sourced GitLab Environment Toolkit used by our QA team for provisioning new systems using Terraform and Ansible.

Using the services calculator, an SA/SAE/AE/CSM can create a scoping issue, and work with an Engagement Manager to iterate and refine the services estimate for a customer. In this issue, we have included additional context to the Implementation Scoping Questions, which can be previewed below

<!--DEBT: below should be embedded from the SSOT locaed <https://gitlab.com/services-calculator/services-calculator.gitlab.io/-/blob/master/make-ps-creates.py>L46-->

Scoping Question Customer Response Example response Rationale for asking
--

- - - -

License level (support for HA requires at least Premium) to-do Premium Validates that the Customer will have the appropriate license needed in order to install, configure and use the requested features.
--

Is this a brand new install of GitLab or do they have an existing instance that they are trying to scale up? to-do New Install If the customer is an existing GitLab deployment and what to move to a new system with stronger availability/stability requirements, we will want to talk about upgrading the current deployment vs standing up a new one then migrating their data.

How many users does the customer expect to have on GitLab when fully implemented? to-do 3,250 The number of users helps us ensure we're implementing an infrastructure that will support the number of users they have, based on GitLab Reference Architectures.
--

Does the customer have a hard requirement for high availability (HA)? to-do Yes - 99.5% availability is our goal Availability requirements help us understand what architecture we will be deploying, which helps us scope the engagement accurately.

Does the customer need secondary site mirroring (Geo) to improve performance for geographically distributed teams or for use for disaster recovery (DR)? to-do Yes - Primary
--

will be in AWS US-east, GEO Secondary in US-west | We need this information to accurately account for time to build and configure a Geo secondary configuration in the estimate. |

| Will the GitLab IaC using terraform and ansible, or CloudFormation on AWS, be acceptable or are there customer-specific IaC requirements? | to-do | Terraform + Ansible preferred | GitLab provides standard Infrastructure as Code (IAC) via terraform and ansible for most deployments. We can offer CloudFormation for AWS, and Helm chart for k8s implementation. If a customer wants different IAC, e.g. Puppet or Chef, then we will need to consider whether we have written this before for a customer or if additional effort needs to be factored in to account for this work. |

| Where is this going to be installed? (AWS, Azure, GCP, on-prem in client's data center) | to-do | AWS | Helps us validate the implementation architecture. AWS deployments are typically less effort than on-prem due to our experience with this cloud and the various services built-in for HA; other clouds (Azure, GCP) might require additional effort. |

| Does the customer plan to install on Kubernetes? If yes: - what k8s implementation will be used (EKS, GKE, AKS, Openshift)? | to-do | Yes, EKS | This changes the deployment architecture (stateless in K8S, stateful in cloud managed storage services) and affects how we scope the implementation engagement. |

| What is the customer's experience level with Kubernetes administration? - Is the customer ok with deploying via Helm? | to-do | Beginners. We're not familiar with Helm | We ask this to ensure customer platform teams have the appropriate required knowledge to deploy and maintain the system using Helm. We don't want customers unable to maintain their deployment after the PS engagement ends. |

| Is there a specific OS required by the customer? (e.g. RedHat Enterprise Linux (RHEL), CENTOS, Ubuntu etc.) | to-do | RHEL | Some deployment tools that we can use only support specific operating systems, which can affect the size of the implementation service engagement. |

| For non-cloud implementations, do you have Redis, nfs etc appliance? | to-do | Yes, we're using NFS for git storage. | Deploying NFS is not recommended as it is in End of Life. Need to know if there are hard requirements for this to provide appropriate guidance in architecture design. |

| Does the customer require the GitLab instance to be air-gapped, i.e. the instance will not be able to connect to the internet? | to-do | Yes, we won't allow the GitLab system to connect directly (or through proxy) to the internet. | Without a way to get to the containers that GitLab CI needs to run build, test, scan, and deploy jobs, the value of GitLab is greatly reduced. If this is the case, we would scope in an effort to build a way to periodically pull in these containers, scan them for security vulnerabilities, and host them internally on a container registry. |

| Does the customer require a proxy to get out to the internet from the GitLab deployment? | to-do | Yes | This is a common approach to reduce security attack vector. To ensure GitLab can function properly, we will help you configure the GitLab system to work with a proxy to ensure GitLab CI jobs can pull containers from registries hosted on the open internet. |

| Will the customer use a publicly trusted certificate or will they use a non-standard Certificate authority? Is the certificate readily available or does it still need to be issued? | to-do | Using publicly trusted certs. Yes we have provisioned the cert already | Publicly trusted certs enable the GitLab standard deployment to be seamless. If there is a non standard Certificate Authority, the deployments are more complex. And if the certs have not yet been acquired, the project could see delays. |

| GitLab, by default, terminates encryption at the load balancer and does not use encryption in transit between GitLab components. Is this encryption method sufficient or will encryption in transit between the components be required? | to-do | Require SSL between all components | If

we need to enable SSL in between all components, the configuration takes longer and needs to be accounted for in the engagement estimate. |

| Does the customer require highly available git storage? | to-do | No, we can use a strong backup/restore policy to reduce maintainability burden | GitLab has a feature called gitaly cluster that replicates git data across many gitlab nodes to ensure the git data is highly available. This feature requires a minimum of 10 additional nodes to the deployment architecture and thus comes with additional maintenance burden. This tradeoff should be considered so we can scope the engagement accurately. |

| For non-cloud and non-kubernetes deployments, what is the hypervisor (e.g. VMWare) that the customer will use for managing VMs? | to-do | N/A - using AWS | This helps us understand if we have the skillset internally to deliver or if we need to work with a partner for fringe hypervisors. |

SECTION: customer-success/professional-services-engineering/engagement-mgmt/scoping-information/readiness.md

:warning: This page is under construction

Scoping Information

General Service Information

A Readiness Assessment aims at providing recommendations to prepare for upcoming growth based on findings from reviewing the Customer's architecture diagram and evaluating their production deployment.

This offering tends to be interesting to Customers who are:

1. Planning for growth - either additional users (e.g. 500 to 2,000) or additional usage (e.g. adopting CI at scale)
1. Deploying a new implementation of GitLab Self Managed and are looking to validate the architecture before onboarding the majority of their users.

A Readiness Assessment is not a service that investigates how to solve a current issue with a GitLab deployment. GitLab support is chartered to help customers fix these kinds of problems.

The outcome of the Health check is a list of findings and recommendations in a Health Check Report. We will not implement the recommendations in the report. This must be scoped in a second SOW or a change order.

Discovery Questions

1. Is your GitLab instance deployed on prem or in a public cloud, which one? (AWS, GCP, Azure Cloud)
1. What is your current GitLab deployment architecture? Can you provide an architecture diagram?
1. Do you have a test environment that is similar in architecture and configuration to your production instance?

- If yes, please describe any differences between the two environments.
 - If no, is there a period of time of low usage on your production instance where the server can be taken offline without being impacted?
1. Has monitoring been established on your production instance to be able to observe error rate, network latency, throughput speeds, disk I/O, and memory/CPU utilization?
 - What logging/observability tool do you have in place (e.g. AWS Cloudwatch, Splunk, GCP Cloud Operations, New Relic)?
 - which components are being monitored?
 1. Are there any known issues with your production server or suspected issues based on end user feedback?
 1. How many user licenses do you have for your GitLab instance? How many active users? If planning on growing your GitLab adoption, how many active users do you anticipate having one year from now?
 1. Are you currently looking to scale out your GitLab implementation to support additional user load? What is your desired end state for your implementation?

Evaluation Method

Our preferred method of analyzing a Customer's deployment architecture is to run GitLab Performance Tool to analyze throughput speeds and errors objectively. However, we do not want to run this on a production GitLab deployment as the load introduced (e.g. dummy data) can have adverse effects on the production GitLab Deployment.

1. Does the Customer have a non-production environment with similar architecture characteristics that we can run GPT on?
 - If yes, move forward with scoping using GPT load method.
1. If not, does the customer have monitoring in place to understand error rate, network latency, throughput speeds, disk I/O, memory/CPU Utilization? Usually this is in the form of observability tools like AWS Cloudwatch, GCP Cloud Operations, New Relic, Splunk, etc
 - If yes, get an understanding of how robust it is. Ask if they have all GitLab components (e.g. Rails nodes, databases, file systems, Redis cache, etc.) monitored.

Typical Estimate Durations

The Readiness Assessment (Health Check) worksheet of the PS Engagement Estimate Templates workbook provides estimates for performing a stand alone readiness assessment via GPT or the User Load Method as part of a new implementation. Note: the estimates are slightly smaller when performed as part of a new implementation PS engagement, since discovery and documentation are handled as part of already defined implementation activities. See the WIP - Implementation worksheet for estimations for a GPT load readiness assessment as part of a PS-led implementation.

SECTION: customer-success/professional-services-engineering/engagement-mgmt/scoping-information/transformational-services.md

General discovery questions

It's helpful to understand the customer's goals and current pain points. The following

questions can be asked of the account team prior to meeting with the customer, or with the customer directly (just make sure to minimize asking the customer to repeat themselves). It's important to uncover these because switching to a new development platform alone will not enable the customer to achieve their goals. Understanding the context of the investment in GitLab is helpful to tailor services to help them meet their needs.

1. What are some of your goals in investing in GitLab? Is this tied to a large initiative like a cloud migration, app modernization or DevOps transformation?
2. What challenges do you have with your current way of working? Are you concerned with how quickly you can deploy changes to production? Do you know where your bottlenecks are?
3. Do you have challenges with functional silos? - is it challenging for developers, IT Ops, QA, and Security to collaborate on a single end to end value stream? Help me understand those challenges a bit better.
4. How do you manage compliance today in your software development lifecycle? What are the biggest challenges you see enforcing compliance?
5. How do you manage your software supply chain (knowing what software is running on all compute resources in your environment). What happens if a vulnerability is detected like log4j? What are the steps to understand how widespread the problem is?
6. Who is responsible for authoring and maintaining the CI/CD pipelines of applications? If it's each application team, do you think there are efficiencies to be gained in standardization of pipelines rather than each team developing a slightly different solution to the same problem?
7. Are there any legacy platform decommission dates (usually driven by license renewals) that are driving your timelines?

SECTION: customer-success/professional-services-engineering/engagement-mgmt/scoping-information/migrations/_index.md

Info for Sales - positioning migrations to SaaS

- GitLab self-managed to SaaS migration

Migration Scoping Details

This page describes scoping migrations from GitLab, Bitbucket Server or GitHub (Enterprise or .com) to a destination GitLab instance (Self-Managed or SaaS). The migrations typically utilize our Congregate automation tool. Migrations from other SCM systems and non-GitLab CI/CD migrations are out of scope for this migration tooling and must be scoped separately.

Using the services calculator, an SA or CSM/CSM can create scoping issue, and work with an Engagement Manager to iterate and refine the services estimate for a customer. In this issue, we have included additional context to the SCM Migration Scoping Questions, which can be previewed below

<!--DEBT: below should be embedded from the SSOT located [| SCM Migration Scoping Question | Customer Response | Example response | Rationale for asking](https://gitlab.com/services-calculator/services-calculator.gitlab.io/-/blob/master/make-ps-creates.pyL25--></p></div><div data-bbox=)

|

| - | - | - | - |

| Source SCM System(s) | to-do | Bitbucket Server, GitLab Self-Managed | GitLab PS has automation to facilitate migration for the most popular source systems. Need to know what systems the data is coming from to accurately scope the time to migrate. |

| Destination GitLab deployment (SaaS, Self-Managed(HA), Self-Managed(single node)) | to-do | SaaS | We ask this to ensure the destination system deployment is strong enough to handle the throughput of data that will be required during migration. |

| Source GitLab version (must be 2 behind latest) | to-do | 13.4 | Migration services leverage many GitLab APIs including our project import/export API. There are specific compatability guidelines documented here |

| Destination GitLab version (must be within 2 minor versions of source) | to-do | 14.1 | Migration services leverage many GitLab APIs including our project import/export API. There are specific compatability guidelines documented here |

| Total number of Users | to-do | BB = 225, GLSM = 775 | Migrating users is a pre-requisite step to migrating the data to ensure data elements are associated properly. This is a discrete task in migration engagements and must be scoped with number of users as an input. |

| Total portfolios (w/ stakeholder rep) | to-do | 6 | We use total portfolios as a proxy metric to identify how much coordination will be required during migration. Each portfolio leader needs to understand and buy into the migration process for things to go smoothly. This coordination time is built into the migration engagement. |

| Total number of git repositories | to-do | BB = 1,234; GL = 4321 | We need to know the total number of repositories to be migrated in order to estimate the number of days of effort. Note: A bitbucket project can contain multiple git repositories. We need the total number of repos. |

|

| Total >5GB GitLab repositories | to-do | BB N/A, GL = 3 | Only applicable for GitLab self-managed migrations to GitLab.com; repos >5GB must be manually migrated due to a 5 GB Cloudflare limitation. Note that we only care about repo size, not the total project size. |

| CI/CD System(s) | to-do | Jenkins | We need to know what CI/CD systems are being used to estimate how much it will take to repoint those pipeline jobs to the new SCM system to help the customer resume IT operations. |

| Total ci/cd jobs? (CI/CD jobs will need cut-over even if not migrated) | to-do | 4567 | The engagement could include repointing CI/CD jobs back to a source repository. If this is the case, we will need to know how many jobs need to be reconfigured. |

| Typical registry size | to-do | 159MB | If registry sizes are unusually big, it could affect the speed of migration. Note: this question only applies to migrations where GitLab is a source system. |

| SSO Identity Provider | to-do | Auth0 | We want to make sure this is already in place prior to migration as it is a foundational to the success of a migration engagement. See here for a full list of supported Identity Providers |

| Source system OS and Version | to-do | Ubuntu v21.10 | If an upgrade of the source system is needed/included prior to the migration, we want to be sure the OS does not need to be upgraded by the customer to support the new version of GitLab. See installation requirements for more details. |

| Source system DB version | to-do | PostgreSQL 13.0 | If an upgrade of the source system is needed/included prior to the migration, we want to be sure the DB does not need to be upgraded by the customer to support the new version of GitLab. See installation requirements for more details. |

Notes:

1. Source Code Management (SCM) Migrations do not include secrets mediation.
1. SCM migrations do not include transitioning CI/CD functionality to use GitLab CI.

GitLab Self-Managed to GitLab.com Migration Notes

- There are specific cases where migrating to SaaS might not be the best recommendation. Here are some considerations when working with the customer to decide on whether or not SaaS is the best solution:

1. Is the customer using something other than GitLab CI? If so, are they able to establish connectivity between their CI tool and GitLab.com? There are occasionally issues that prevent them from using GitLab and their CI/CD tools, i.e. the customer is not able to convince their network security team to adjust firewall configurations to be able to use their CI/CD tool behind their firewall with GitLab SaaS.

1. If the customer has a very large number of projects, this could become a change management challenge as well as be cost prohibitive to do the migration. e.g. for one customer recently, who has been having performance issues in their self-managed instance, it was going to take \$400k and 150+ days to migrate them to SaaS (1600 users and 20k+ projects). GitLab Dedicated might be a better option for some customers who are using Ultimate.

1. The customer may be using some self-managed features that they are insistent on continuing to use that aren't available on SaaS (e.g. server hooks). See Differences Between Self-Managed and SaaS for more information.

1. The customer may have very large GitLab projects that would be difficult to migrate to SaaS, and they would need to pay for additional storage consumption in order to use SaaS; plus there are known git and CI performance issues when customers have very large monorepos.

- Migrating from Self-Managed to SaaS has additional information, rough order magnitude estimates, and common customer questions

- see Congregate Features Matrix to see a comparison of GitLab ui migration vs Congregate migration and gain a better understanding of which features get migrated and which don't.

- Migrations from one GitLab instance to another using Congregate require that the group and project structure of the source system is retained during migration.

- scoping uses our standard Automated Migration PS Engagement Estimate Template. For very small migrations with less than 30 repositories, use the Manual Migration Estimate Template to compare the cost of a manual migration to the cost of automated migration

- SaaS Discovery, SSO configuration and Security configuration activities are typically added to migrations to SaaS. See Services Calculator activities for more details:

- saasdiscoveryremote

- saasssoconfigurationremote

- saassecurityconfigurationremote

- see TEMPLATE Professional Services Presentation for a summary of our migration approach and which features get migrated or not. This also includes a description of our SaaS Discovery, SSO configuration and Security configuration activities, which we often add for new SaaS customers.

GitLab Self-Managed to GitLab Self-Managed

- You can use Congregate but you also have other options at your disposal:

- Backup and restore using rake-tasks. This enables you to restore all of the data from one instance of GitLab to another. This can be used to shorten the migration period, but will cause complete downtime as the source instance will need to be locked during the time it takes to backup. Also, this could take many hours to days depending on the size of the data.

- Geo Replication. This method allows you to setup the new GitLab instance as a Geo secondary. The data will sync over the course of time. Then you can perform a failover to make the new instance primary. This tends to be more complex and requires a more specialized skillset than the backup/restore approach.

GitLab Self-Managed to GitLab Dedicated Instance

- As with self-managed to self-managed migrations, self-managed to dedicated instance migrations can be performed using Congregate, a backup and restore approach, or Geo replication. If the Congregate option is used, projects can be migrated in waves of up to 500.
- Migrating groups with projects is a Beta feature could also be used to migrate to dedicated instance. Dedicated instance needs to be on GitLab version 15.10 or later. Please review linked documentation to understand limits and requirements of this method.

GitHub Source

GitHub Enterprise to GitLab Self-Managed

- This is one of our strongest combinations of source and destination systems. We have migrated 7,000+ projects per wave in our best case.
- If minimizing the transition period is a concern, we will want to maximize the number of projects per wave.
 - To do this, we need to have full control over the GitHub API Rate limit setting.
 - We also want to ensure there is a strong enough network connection between the GHE and GitLab SM instance to support moving the data. We don't have a hard number, but ask them about moving hundreds of GB over a few hours across lots of connections. Their network team should be able to answer.
- If the customer wants to reorganize their groups/project structure, we can support this (see below common customer requests).
- Make sure the customer acknowledges that they need to have email addresses public, not private for any migration to work properly.
- GitLab self managed should be on 13.7+ or later to take advantage of an API change that allows us to specify a specific github hostname in the import call (rather than defaulting to github.com).

GitHub Enterprise to GitLab SaaS

- The destination system rate limit will be the limiting factor in these migrations. We have no control over gitlab.com rate limit so these will need to be scoped at 200-300 per wave.
- GitLab self managed should be on 13.7+ or later to take advantage of an API change that allows us to specify a specific github hostname in the import call (rather than defaulting to github.com).
- Make sure the customer acknowledges that they need to have email addresses public, not private for any migration to work properly.

Github.com to GitLab Self-Managed

- The Github.com API rate limit is very low per user so we workaround by using multiple userIDs provided by the customer.
- The total projects migrated per day should not exceed 200 until we deliver this to a customer

at scale. We've done it at small scale in terms of number of projects. But some of those projects were gigantic (70K+ Pull Requests).

- Make sure the customer acknowledges that they need to have email addresses public, not private for any migration to work properly.

Github.com to GitLab SaaS

- We are limited by both the source and destination system rate limits.
- Make sure the customer acknowledges that they need to have email addresses public, not private for any migration to work properly.

Bitbucket Source

Note: A project on bitbucket is equivalent to a GitLab group. A Repository on Bitbucket is equivalent to a GitLab project.

Bitbucket Server to GitLab Self-Managed

- Theoretically, migrations with this pair of source/destination should be able to be scaled as high as GHE to GL Self-Managed. Its safe to increase the projects per wave to 1,000.
- If minimizing the transition period is a concern, we will want to maximize the number of projects per wave.
 - To do this, we need to have full control over the BitBucket API Rate limit setting.
 - We also want to ensure there is a strong enough network connection between the BitBucket Server and GitLab SM instance to support moving the data. We don't have a hard number, but ask them about moving hundreds of GB over a few hours across lots of connections. Their network team should be able to answer.
- If the customer wants to reorganize their groups/project structure, we can support this (see below common customer requests).

Bitbucket Cloud to GitLab Self-Managed

- GitLab does not have an API to initiate an import from bitbucket cloud currently. Automated migrations are not possible.
- Can position a teach a customer to fish advisory approach that uses the BB cloud import UI to help with importing.

Bitbucket Server to GitLab SaaS

- We will be limited by the rate limit on GitLab SaaS, which is why our scoping is low (e.g. 200 projects per wave)

Bitbucket Cloud to GitLab SaaS

- GitLab does not have an API to initiate an import from bitbucket cloud currently. Automated migrations are not possible.
- Can position a teach a customer to fish advisory approach that uses the BB cloud import UI to help with importing.

Team Foundation Server (TFS)/Azure DevOps (ADO) to GitLab

Azure DevOps (formerly named Team Foundation Server) contains more than a source code repository, so additional questions need to be asked while scoping out an ADO migration. Five common components in ADO are the following:

- Azure Boards: Set of Agile tools to plan, structure and manage work through a backlog. Equivalent to GitLab Issues or JIRA tickets.
- Azure Repos: Version control system. Supports Git or Team Foundation Version Control (TFVC).
- Azure Pipelines: Build and release services to support continuous integration and delivery of an application. Equivalent to GitLab CI/CD.
- Azure Test Plans: Several tools to test an application, including manual/exploratory testing and continuous testing.
- Azure Artifacts: Service to store and share packages such as Maven, npm, NuGet, generic and others from public and private sources. Equivalent to GitLab Package Registry.

| Question | Answer | Sample Answer | Rationale for asking|

| ---- | ---- | ---- | ---- |

| General ||||

| Is it SaaS or self-hosted?|| Azure DevOps (SaaS) | If ADO Server (self-hosted), then we need to know software version and other details (i.e. SQL version etc), this will help to pick the right migration approach. |

| How many organizations do you have that need to be migrated? | | 1 | Answer to this question will help to understand the current state, amount of required effort and contribute to future architecture of GitLab. |

| How many projects per organization? | | 500 | Answer to this questions will help to estimate amount of time and approach for migration. |

| What is the ratio project/repo? | | 1:1 | Based on answer to this question additional advisory services to refactor/restructure might be required |

| Azure Boards ||||

| Are you using workitems in ADO? || Yes | If workitems need to be retained, then additional migration and/or advisory activities need to be added to the engagement. Note: there are features in ADO workitems (e.g. custom fields and workflows) that are not supported by GitLab issues. During scoping, make sure the customer is aware of these differences. |

| What template do you use? ||Scrum, Agile, SCCM | To understand the current state and ensure GitLab's Issues support all the capabilities. |

| What customizations do you use in your boards? ||Extra swimming lanes, custom fields etc|Based on this answer we may require additional advisory services to discuss what is possible / not possible and propose alternatives. |

| Azure Repos ||||

| Are you using Git or TFVC for your SCM? | | Git | This will influence how we interact with the ADO server and determine if a conversion to Git is necessary |

| How many code repositories? || 500 | This will affect the decision on general approaches listed above |

| Are you using branches in your code repositories? | | Yes | If the answer is no for some of the repositories, the customer will have to choose between converting specific folders in the repository to branches to retain history or accept a flat file migration of the repository with no history |

| Do you need to retain history in your code repositories? | | Yes | If the answer is no and the customer is using ADO, then we do not need to convert the repos to Git. |

| Azure Pipelines ||||

| Are you using ADO to build your software? | | Yes | This will add CI/CD

consulting/transformation activities to the engagement if the answer is yes |

| How many builds/build templates are used per code repository? || 1 | This is a gauge of complexity. Sometimes a code repository can contain several different build definitions |

| Are multiple solution (.sln) or project (.csproj) files building the same packages? || Yes |

If yes, this can require advisory services to refactor to those solution/project files to work with GitLab pipelines |

| How many build servers do you use? | | 1 | We usually convert a build server to something more ephemeral so the more build servers in use, the more development is required to transition them to something more ephemeral |

| Are there any specific flags used in your build process? If so, what? | | Yes | This shows the customer is measuring this data to be used during the transition process. |

| Do you use classic releases? || No | If the answer is yes then additional effort might be required to educate on YAML (pipeline as code) concept |

| Do you use UI build definitions? || No | If yes then this will add CI/CD consulting/transformation activities |

| Do you use YAML for all your definitions (build / release)? || Yes | If "Yes" is the answer to this question then SOW might include additional effort and advisory services (i.e. CI/CD consulting/transformation activities)|

| Do you use extensions from the marketplace that are part of your build/test/release process? Provide the list of extensions (if the answer is "Yes") || Yes. ARM-ttk extension for linting templates. |If yes then we need to ensure that we have equivalent in GitLab for this (or propose alternative solution)|

| What type of agents do you use? || Self-Hosted, Microsoft-hosted (i.e. windows-latest, ubuntu-latest etc) |Based on this answers we will make a decision what runner architecture to propose and make sure GitLab supports it and how much effort it requires to achieve similar state|

| Do they have special requirements (i.e. network connectivity, required components for build/test/release process)? ||Yes, need Visual Studio 2017 capabilities (i.e. legacy software)| Based on the answer we will make a decision if custom runner images are needed (i.e. to install capabilities) |

| Are there any plans in the future to use/add anything special in the CI/CD? || GPU optimized agent | Based on the answer we will make a decision on GitLab Runners to avoid potential limits in the future |

| Azure Test Plans ||||

| Do you use test plans and do you need similar functionality in GitLab? || Yes |Based on the answer we need to propose equivalent feature in GitLab|

| Azure Artifacts ||||

| Do you use artifact feeds? || Yes |To understand the current state|

| What type of feeds? || Maven, npm, NuGet | To understand the current state an